

Тема 3. Встроенный модуль path. Относительные пути. Константа __dirname



Хекслет колледж

Цель занятия:

Научить студентов правильно работать с путями к файлам и папкам в Node.js - независимо от операционной системы.

Учебные вопросы:

1. Проблема путей - зачем нужен модуль path
2. Абсолютные и относительные пути
3. Константы `__dirname` и `__filename`
4. Основные методы модуля path
5. Практика - строим пути правильно

1. Проблема путей - зачем нужен модуль path

Разница между путями в Windows и Linux

Когда программа работает с файлами, ей нужно знать где они находятся - то есть путь к файлу.

Проблема в том, что Windows и Linux записывают пути по-разному:

	Windows	Linux / macOS
Разделитель	\ (обратный слэш)	/ (прямой слэш)
Пример пути	C:\project\data\file.txt	/home/user/project/data/file.txt

Если написать путь вручную под Windows:

```
const filePath = 'data\\file.txt';
```

На Linux этот код не будет работать корректно - разделитель другой.

Почему нельзя просто писать пути вручную?

Рассмотрим пример - допустим, мы хотим обратиться к файлу file.txt в папке data:

```
// ❌ Плохо – жёстко прописанный путь, работает только на Windows  
const filePath = 'C:\\project\\data\\file.txt';
```

```
// ❌ Плохо – может сломаться при смене папки проекта  
const filePath = 'data/file.txt';
```

Проблемы ручных путей:

- Зависят от операционной системы
- Зависят от того, из какой папки запущена программа
- Легко допустить ошибку при написании

Как модуль path решает эту проблему?

Модуль **path** автоматически использует правильный разделитель для той системы, на которой запущена программа. Нам не нужно думать о слэшах - **path** сделает всё сам:

```
const filePath = path.join('data', 'file.txt');  
console.log(filePath);  
// Windows: data\file.txt  
// Linux:   data/file.txt
```

Один и тот же код - правильный результат на любой системе.

Свойство `path.sep`

Если интересно какой разделитель используется на текущей системе - можно проверить:

```
const path = require('path');  
  
console.log(path.sep);  
// Windows: \  
// Linux:   /
```

Итог:

Проблема	Пути в Windows и Linux записываются по-разному
Ручные пути	Зависят от ОС, ненадёжны
Решение	Модуль path — строит пути правильно на любой системе
Подключение	<code>const path = require('path')</code>

2. Абсолютные и относительные пути

Абсолютный путь - это полный путь к файлу, начиная от корня файловой системы. Он всегда указывает на одно и то же место, независимо от того откуда запущена программа.

Примеры:

Windows: `C:\Users\user\project\data\file.txt`

Linux: `/home/user/project/data/file.txt`

В Node.js:

```
const filePath = "C:\\Users\\user\\project\\data\\file.txt"
```

Минус: абсолютный путь привязан к конкретному компьютеру. Если перенести проект на другой компьютер или сервер - путь сломается.

Относительный путь - это путь относительно текущего расположения. Он не начинается от корня, а отсчитывается от какой-то точки.

Запись	Значение
<code>./</code>	Текущая папка
<code>../</code>	Папка на уровень выше
<code>../..</code>	Папка на два уровня выше

Примеры:

```
'./data/file.txt'    // файл в папке data рядом с текущим файлом  
 '../file.txt'       // файл в папке на уровень выше  
 './file.txt'        // файл рядом с текущим файлом
```

Минус: относительный путь зависит от того, из какой папки запущена программа. Если запустить программу из другой папки - путь сломается.

Проблема относительных путей в Node.js

Рассмотрим структуру проекта:

```
▼ PROJECT
  ▼ data
    ≡ file.txt
  JS index.js
```

```
data > ≡ file.txt
1      Hello!!!
```

Кажется, что можно написать так:

```
const fs = require('fs');  
const data = fs.readFileSync('./data/file.txt', 'utf8');  
console.log(data) // Hello!!!
```

Это работает, если запускать из папки
project:

```
PS C:\Users\user\Documents\nodeproject\project> node index.js  
Hello!!!
```

Но если запустить из другой папки:

```
PS C:\Users\user\Documents\nodeproject> node project/index.js
node:fs:441
    return binding.readFileUtf8(path, stringToFlags(options.flag));
    ^
Error: ENOENT: no such file or directory, open 'C:\Users\user\Documer
```

Относительный путь `./` отсчитывается от папки откуда запущена программа, а не от папки где находится файл.

Решение - `__dirname`

Чтобы путь всегда работал независимо от того откуда запущена программа - используют `__dirname`.
Это абсолютный путь к папке текущего файла:

```
const path = require('path');  
const fs = require('fs');  
  
const filePath = path.join(__dirname, 'data', 'file.txt');  
const data = fs.readFileSync(filePath, 'utf8');  
console.log(data)
```

Теперь путь всегда строится от папки где находится `index.js` - неважно откуда запущена программа.

Итог:

	Абсолютный путь	Относительный путь
Начинается от	Корня файловой системы	Текущей папки запуска
Пример	C:\project\file.txt	./data/file.txt
Плюс	Всегда точный	Короткий, переносимый
Минус	Привязан к компьютеру	Зависит от места запуска
Решение в Node.js	<code>path.join(__dirname, ...)</code>	<code>path.join(__dirname, ...)</code>

3. Константы `__dirname` и `__filename`

`__dirname` и `__filename` - это специальные константы, которые Node.js автоматически создаёт в каждом файле.

Их не нужно импортировать - они просто есть.

```
console.log(__dirname); // полный путь к папке текущего файла
console.log(__filename); // полный путь к текущему файлу
```

__dirname

__dirname содержит абсолютный путь к папке, в которой находится текущий файл. Пример - файл находится по пути C:\project\index.js:

```
console.log(__dirname);  
// Windows: C:\project  
// Linux:    /home/user/project
```

Главное свойство - `__dirname` всегда указывает на папку файла, независимо от того откуда запущена программа:

- запуск из любой папки - результат одинаковый

`node C:\project\index.js`

`node project\index.js`

`node index.js`

Во всех трёх случаях `__dirname` вернёт `C:\project`.

__filename

__filename содержит абсолютный путь к самому файлу - папка плюс имя файла:

```
console.log(__filename);  
// Windows: C:\project\index.js  
// Linux:    /home/user/project/index.js
```

Разница между __dirname и __filename

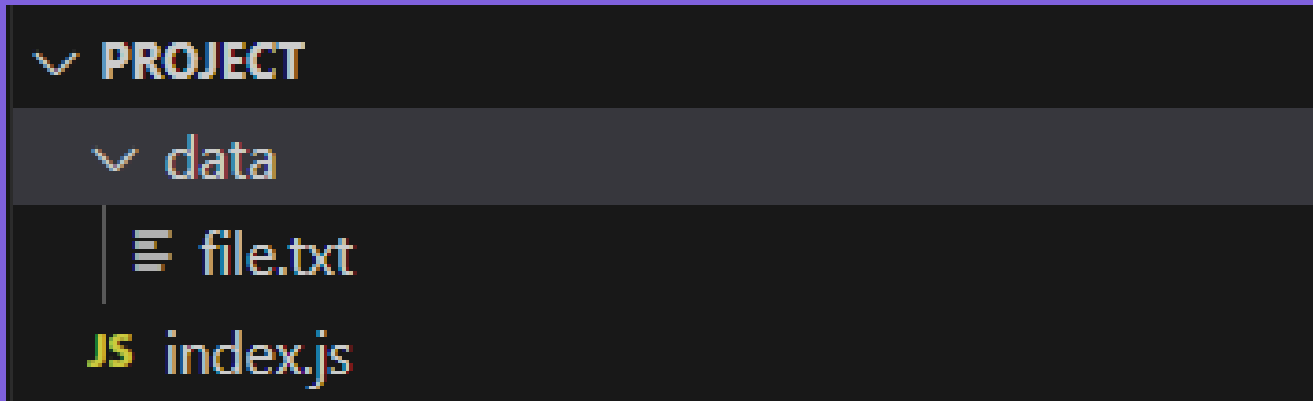
```
console.log(__dirname); // C:\project  
console.log(__filename); // C:\project\index.js
```

Простыми словами:

__dirname - папка где лежит файл

__filename - папка плюс имя самого файла

Зачем они нужны - практический пример
Самое частое применение `__dirname` - строить надёжные пути к файлам через `path.join()`:
Структура проекта:



```
const path = require('path');

// ✗ Ненадёжно – зависит от места запуска
// const filePath = './data/file.txt';

// ✔ Надёжно – всегда правильный путь
const filePath = path.join(__dirname, 'data', 'file.txt');

console.log(filePath);
// Windows: C:\project\data\file.txt
// Linux:    /home/user/project/data/file.txt
```


Итог:

<code>__dirname</code>	Абсолютный путь к папке текущего файла
<code>__filename</code>	Абсолютный путь к текущему файлу
Импорт	Не нужен — доступны в каждом файле
Главное применение	<code>path.join(__dirname, ...)</code> для надёжных путей

4. Основные методы модуля path

`path.join()` - соединить части пути

Соединяет несколько частей в один путь, автоматически подставляя правильный разделитель:

```
const filePath = path.join('project', 'data', 'file.txt');
console.log(filePath);
// Windows: project\data\file.txt
// Linux:   project/data/file.txt
```

C __dirname:

```
const filePath = path.join(__dirname, 'data', 'file.txt');  
console.log(filePath);  
// C:\project\data\file.txt
```

path.join() - самый используемый метод.
Применяется каждый раз когда нужно
построить путь к файлу.

`path.resolve()` - получить абсолютный путь
Строит **абсолютный** путь, обрабатывая
аргументы справа налево до тех пор, пока не
получит абсолютный путь:

```
console.log(path.resolve('data', 'file.txt'));  
// C:\project\data\file.txt (добавляет текущую папку автоматически)  
  
console.log(path.resolve('/home', 'user', 'file.txt'));  
// /home/user/file.txt
```

Если абсолютный путь не получается из
аргументов - автоматически подставляет текущую
рабочую папку

`path.basename()` - имя файла

Возвращает имя файла из полного пути:

```
const filePath = 'C:\\project\\data\\file.txt';  
console.log(path.basename(filePath));           // file.txt  
console.log(path.basename(filePath, '.txt'));    // file (без расширения)
```

`path.dirname()` - папка файла

Возвращает путь к папке, в которой находится файл:

```
const filePath = 'C:\\project\\data\\file.txt';  
console.log(path.dirname(filePath)); // C:\project\data
```

`path.extname()` - расширение файла

Возвращает расширение файла включая точку:

```
console.log(path.extname('file.txt'));    // .txt
console.log(path.extname('image.png'));   // .png
console.log(path.extname('index.js'));    // .js
console.log(path.extname('archive'));     // '' (нет расширения)
```

`path.parse()` - разобрать путь на части

Разбирает путь на отдельные составляющие и возвращает объект:

```
const filePath = 'C:\\project\\data\\file.txt';  
console.log(path.parse(filePath));
```

```
{  
  root: 'C:\\',  
  dir: 'C:\\project\\data',  
  base: 'file.txt',  
  ext: '.txt',  
  name: 'file'  
}
```


Итог:

Метод	Назначение	Пример результата
<code>path.join()</code>	Соединить части пути	<code>data\file.txt</code>
<code>path.resolve()</code>	Получить абсолютный путь	<code>C:\project\data\file.txt</code>
<code>path.basename()</code>	Имя файла	<code>file.txt</code>
<code>path.dirname()</code>	Папка файла	<code>C:\project\data</code>
<code>path.extname()</code>	Расширение файла	<code>.txt</code>
<code>path.parse()</code>	Разобрать путь на части	<code>{ root, dir, base, ext, name }</code>

5. Практика - строим пути правильно

Типичные ошибки при работе с путями

1. Жёстко прописанный абсолютный путь

```
// ❌ Плохо – работает только на одном конкретном компьютере  
const filePath = 'C:\\Users\\Алексей\\project\\data\\file.txt';
```

На другом компьютере или сервере этот путь не существует - программа сломается.

2. Ручная склейка пути через +

```
// ❌ Плохо – разделитель может быть неправильным  
const filePath = __dirname + '/data/' + 'file.txt';
```

На Windows разделитель \, поэтому / может вызвать проблемы. Кроме того легко допустить лишний или пропущенный слэш.

3. Относительный путь без __dirname

```
// ✗ Плохо – разделитель может быть неправильным  
const filePath = __dirname + '/data/' + 'file.txt';
```

Работает только если запускать программу из правильной папки. При запуске из другой папки - путь сломается.

4. Лишние или пропущенные слэши

```
// ❌ Лишний слэш в начале  
const filePath = path.join(__dirname, '/data/file.txt');
```

```
// ❌ Лишний слэш в конце папки  
const filePath = path.join(__dirname + '/', 'data', 'file.txt');
```

`path.join()` сам расставляет разделители -
не нужно добавлять их вручную.

Правильное построение пути

Основное правило:

Всегда строить пути через
`path.join(__dirname, ...)`:

```
const path = require('path');
```

```
//  Правильно – надёжно на любой системе и из любой папки  
const filePath = path.join(__dirname, 'data', 'file.txt');
```

Примеры для разных структур проекта

Файл в той же папке:

```
// project/  
// └─ index.js  
// └─ file.txt  
const filePath = path.join(__dirname, 'file.txt');
```

Файл во вложенной папке:

```
// project/  
// |— index.js  
// |— data/  
//   |— file.txt  
const filePath = path.join(__dirname, 'data', 'file.txt');
```


Файл на уровень выше:

```
// project/  
// └─ data/  
//   └─ file.txt  
// └─ src/  
//     └─ index.js  
// index.js находится в src/  
const filePath = path.join(__dirname, '..', 'data', 'file.txt');
```

Глубоко вложенный файл:

```
// project/  
// └─ index.js  
//   └─ config/  
//       └─ settings/  
//           └─ app.json  
const configPath = path.join(__dirname, 'config', 'settings', 'app.json');
```

Итог:

✗ Не использовать

Жёстко прописанные абсолютные пути

✗ Не использовать

Склейку через `+`

✗ Не использовать

`./` без `__dirname`

✗ Не использовать

Ручные слэши внутри `path.join()`

✓ Использовать

`path.join(__dirname, 'папка', 'файл')`

Выводы:

- В Windows разделитель путей \, в Linux - / - писать пути вручную ненадёжно
- Модуль path автоматически использует правильный разделитель для любой системы
- Абсолютный путь начинается от корня файловой системы - привязан к конкретному компьютеру
- Относительный путь отсчитывается от папки запуска программы - может сломаться при запуске из другой папки
- `__dirname` - абсолютный путь к папке текущего файла, доступен без импорта
- `__filename` - абсолютный путь к самому файлу, доступен без импорта
- `__dirname` всегда указывает на папку файла - независимо от того откуда запущена программа
- `path.join()` - склеивает части пути с правильным разделителем
- `path.resolve()` - всегда возвращает абсолютный путь
- Главное правило - всегда строить пути через `path.join(__dirname, ...)`
- Нельзя склеивать пути через + и вручную добавлять слэши внутри `path.join()`

Контрольные вопросы

1. Почему нельзя писать пути к файлам вручную через / или \?
2. Чем абсолютный путь отличается от относительного?
3. Что хранит `__dirname`? Чем отличается от `__filename`?
4. Почему использование `./` без `__dirname` ненадёжно?
5. Что делает метод `path.join()`? Приведите пример.
6. Чем `path.join()` отличается от `path.resolve()`?
7. Какой метод вернёт только имя файла из полного пути?
8. Какой метод вернёт расширение файла?
9. Что вернёт `path.parse()`? Какие поля будут в результате?
10. Как правильно построить путь к файлу `file.txt` в папке `data`, которая находится рядом с текущим файлом?